

ADVANCED NETWORK SECURITY AND ARCHITECTURES

METI

Report Module 1 [EN]

Authors:

José Oliveira (103771)

Afonso Mendes (116024)

Tiago Videira (116069)

jose.novais.oliveira@tecnico.ulisboa.pt

afonsofmenDES@tecnico.ulisboa.pt

tiago.g.videira@tecnico.ulisboa.pt

Group 3

Contents

1	Introduction	3
2	Network attacks and countermeasures	4
2.1	Objective	4
2.2	MAC Table Overflow	5
2.2.1	Attack and Countermeasure	5
2.2.2	Switch Comparison	5
2.2.3	Feasibility	6
2.3	DHCP spoofing	6
2.3.1	Attack and Countermeasure	6
2.3.2	AI Prediction	7
2.3.3	Switch Comparison	7
2.3.4	Feasibility	7
2.4	ARP poisoning (MitM)	7
2.4.1	Network Topology	7
2.4.2	Attack and Countermeasure	8
2.4.3	Defense Statistics	9
2.4.4	AI Configuration vs Experimental Setup	9
2.4.5	Switch Comparison	9
2.4.6	Feasibility	10
2.5	DNS spoofing	10
2.6	RIP Poisoning	10
2.6.1	Network Topology	10
2.6.2	Attack and Countermeasure	10
2.6.3	Effectiveness of the Attack	12
2.6.4	AI Prediction vs Experimental Results	13
2.6.5	Feasibility	13
3	Firewalls	14
3.1	Classical firewalls versus Zone-Based Policy Firewalls	14
3.1.1	Firewall Validation	14
3.1.2	Classical Firewall (CBAC)	14
3.1.3	ZBPF	14
3.2	NMAP Operation	15
3.2.1	Host Discovery	15
3.2.2	Port Scanning	15
3.3	CBAC vs ZBPF	15
3.4	Protecting a Campus Network using ZBPF	16
3.4.1	Network Topology	16
3.4.2	Firewall Configuration	16
3.4.3	Testing and Validation	17
3.4.4	Conclusion	19
3.5	Defense against DoS attacks	19

4	AAA	20
4.1	AAA with TACACS+	20
4.1.1	AAA Configuration	20
4.1.2	Authentication, Authorization and Accounting	21
4.1.3	Password Configuration	23
4.1.4	Wireshark Analysis	24
4.1.5	Benefits of Server-Based AAA	24
4.2	AAA with Radius	25
4.3	802.1X Authentication	26
4.3.1	Wireshark Analysis of EAP-PEAP Authentication	26
4.3.2	Security Considerations	28
4.3.3	Comparison Between AI Explanation and Observed Traffic	28
5	Conclusion	30
6	Appendix	31

1 Introduction

FAZER APENAS NO FIM DOS 3 LABS

2 Network attacks and countermeasures

2.1 Objective

The main objective of this laboratory work is to study and experimentally evaluate a set of common network attacks and their corresponding countermeasures in a controlled environment using GNS3.

In particular, the following attacks are addressed:

- **MAC table overflow:** to understand how a switch can be forced to behave as a hub by saturating its MAC address table, and to evaluate mitigation mechanisms such as port security.
- **DHCP spoofing:** to demonstrate how a rogue DHCP server can provide malicious network configurations to clients, and to analyze the effectiveness of DHCP snooping as a countermeasure.
- **ARP poisoning (Man-in-the-Middle):** to perform a MitM attack by manipulating ARP tables, and to study the use of Dynamic ARP Inspection (DAI) for protection.
- **DNS spoofing:** to analyze how a victim can be redirected to a fake web server through DNS manipulation combined with ARP poisoning.
- **RIP poisoning:** to investigate how routing tables can be corrupted using forged RIP messages, leading to traffic redirection, and to explore authentication-based defenses.

Additionally, this work aims to:

- Analyze network behavior through packet captures using Wireshark;
- Interpret device-level information using Cisco IOS commands;
- Evaluate the practical feasibility and limitations of each attack;
- Compare experimental results with theoretical expectations.

2.2 MAC Table Overflow

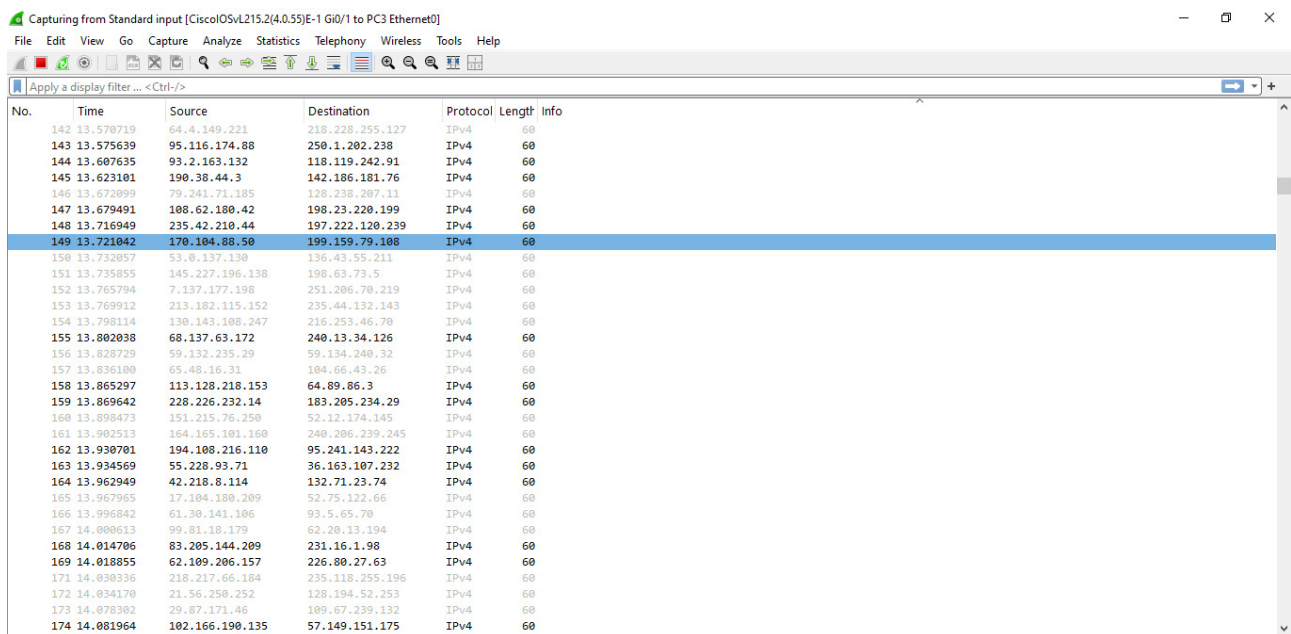
2.2.1 Attack and Countermeasure

The MAC table overflow attack was performed using the `camov.py` script, generating a large number of frames with different spoofed MAC addresses.

As observed in Wireshark (Figure 1), a high volume of traffic with varying source addresses was captured, indicating the flooding behavior of the attack.

The switch behavior was analyzed using `show mac address-table`.

To mitigate the attack, port security was configured, limiting the number of MAC addresses per port and preventing the insertion of multiple spoofed entries.



No.	Time	Source	Destination	Protocol	Length	Info
142	13.578719	64.4.149.221	218.228.255.127	IPv4	60	
143	13.575639	95.116.174.88	250.1.202.238	IPv4	60	
144	13.607635	93.2.163.132	118.119.242.91	IPv4	60	
145	13.623101	190.38.44.3	142.186.181.76	IPv4	60	
146	13.672099	79.241.71.185	128.238.207.11	IPv4	60	
147	13.679491	108.62.180.42	198.23.220.199	IPv4	60	
148	13.716949	235.42.210.44	197.222.120.239	IPv4	60	
149	13.721042	170.104.88.50	199.159.79.108	IPv4	60	
150	13.732057	33.0.137.130	136.43.59.211	IPv4	60	
151	13.735855	145.227.196.138	198.63.73.5	IPv4	60	
152	13.765794	7.137.177.198	251.206.70.219	IPv4	60	
153	13.769912	213.182.115.152	235.44.132.143	IPv4	60	
154	13.798114	130.143.108.247	216.253.46.70	IPv4	60	
155	13.802038	68.137.63.172	240.13.34.126	IPv4	60	
156	13.828729	59.132.235.29	59.134.240.32	IPv4	60	
157	13.836100	65.48.16.31	104.66.43.26	IPv4	60	
158	13.865297	113.128.218.153	64.89.86.3	IPv4	60	
159	13.869642	228.226.232.14	183.205.234.29	IPv4	60	
160	13.898473	151.215.76.250	52.12.174.145	IPv4	60	
161	13.902513	164.165.101.160	240.206.230.245	IPv4	60	
162	13.930701	194.108.216.110	95.241.143.222	IPv4	60	
163	13.934569	55.228.93.71	36.163.107.232	IPv4	60	
164	13.962949	42.218.8.114	132.71.23.74	IPv4	60	
165	13.967965	17.104.180.209	52.75.122.66	IPv4	60	
166	13.996842	61.30.141.106	93.5.65.70	IPv4	60	
167	14.000613	99.81.18.179	62.20.13.194	IPv4	60	
168	14.014706	83.205.144.209	231.16.1.98	IPv4	60	
169	14.018855	62.109.206.157	226.80.27.63	IPv4	60	
171	14.030336	218.217.66.104	235.118.255.196	IPv4	60	
172	14.034170	21.56.250.252	128.194.52.253	IPv4	60	
173	14.078302	29.87.171.46	109.67.239.132	IPv4	60	
174	14.081964	102.166.190.135	57.149.151.175	IPv4	60	

Figure 1: Wireshark capture during MAC table overflow attack

2.2.2 Switch Comparison

Cisco Catalyst 2960 supports port security, allowing the configuration of a maximum number of MAC addresses per port and defining actions when violations occur (e.g., restrict or shutdown). This makes it possible to prevent MAC table overflow attacks.

Link: www.cisco.com/c/en/us/products/switches/catalyst-2960-series-switches/index.html

In contrast, the TP-Link TL-SF1005D is an unmanaged switch that does not support port security or any traffic control mechanisms, as it lacks a configuration interface. Therefore, it cannot prevent MAC flooding attacks.

Link: www.tp-link.com/en/home-networking/soho-switch/tl-sf1005d/

2.2.3 Feasibility

The objective of this attack is to flood the switch MAC address table, forcing the switch to behave like a hub. In that state, frames may be forwarded to all ports, allowing the attacker to listen to conversations between other hosts.

However, in practice, the attack may be difficult to execute successfully on modern switches because their MAC address tables are large. Additionally, the high amount of generated traffic can cause network instability or a denial-of-service condition, instead of allowing the attacker to reliably capture useful traffic.

Therefore, although the attack is theoretically possible, its practical feasibility is limited and depends on the switch capacity, configuration, and security features.

2.3 DHCP spoofing

2.3.1 Attack and Countermeasure

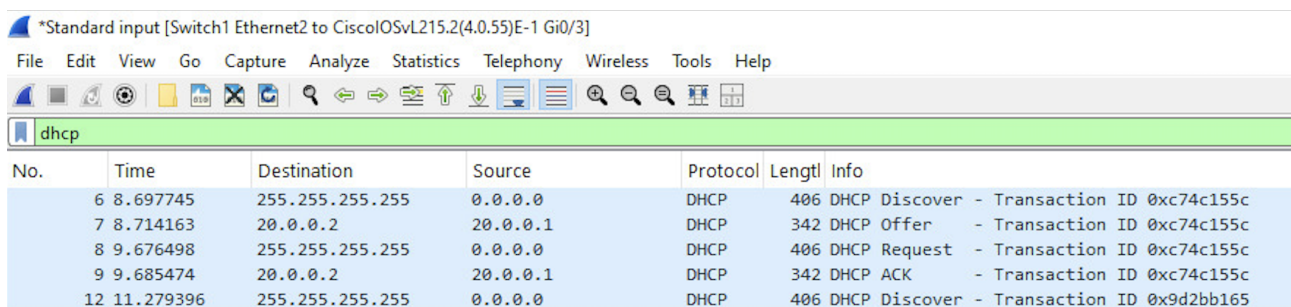
The DHCP spoofing attack was performed by introducing a rogue DHCP server that responds to DHCP Discover messages with malicious configuration parameters.

As observed in Wireshark (Figure 2), multiple DHCP Offer messages were received for the same transaction ID, indicating a race condition between the legitimate and the rogue DHCP servers. The victim accepts the first Offer received, which may result in a malicious gateway configuration provided by the attacker.

20	20.480048	20.0.0.100	20.0.0.200	DHCP	338 DHCP Offer	- Transaction ID 0x2c5ff044
21	20.482689	20.0.0.2	20.0.0.1	DHCP	342 DHCP Offer	- Transaction ID 0x2c5ff044
23	21.447930	255.255.255.255	0.0.0.0	DHCP	406 DHCP Request	- Transaction ID 0x2c5ff044
24	21.493701	20.0.0.100	20.0.0.200	DHCP	338 DHCP ACK	- Transaction ID 0x2c5ff044

Figure 2: Wireshark capture showing multiple DHCP Offer messages (attack)

After enabling DHCP snooping and configuring the port connected to the legitimate DHCP server as trusted, only one DHCP Offer was observed (Figure 3). The rogue DHCP responses were blocked by the switch, and the client received a valid configuration. This confirms that DHCP snooping effectively mitigates the attack.



The image shows a Wireshark capture window titled '*Standard input [Switch1 Ethernet2 to CiscoIOSvL215.2(4.0.55)E-1 Gi0/3]'. The packet list shows five DHCP packets. The packet details pane shows the selected packet (No. 7) as a DHCP Offer with Transaction ID 0xc74c155c. The packet bytes pane shows the DHCP Offer structure.

No.	Time	Destination	Source	Protocol	Length	Info
6	8.697745	255.255.255.255	0.0.0.0	DHCP	406	DHCP Discover - Transaction ID 0xc74c155c
7	8.714163	20.0.0.2	20.0.0.1	DHCP	342	DHCP Offer - Transaction ID 0xc74c155c
8	9.676498	255.255.255.255	0.0.0.0	DHCP	406	DHCP Request - Transaction ID 0xc74c155c
9	9.685474	20.0.0.2	20.0.0.1	DHCP	342	DHCP ACK - Transaction ID 0xc74c155c
12	11.279396	255.255.255.255	0.0.0.0	DHCP	406	DHCP Discover - Transaction ID 0x9d2bb165

Figure 3: Wireshark capture with DHCP snooping enabled (only legitimate Offer)

2.3.2 AI Prediction

The AI predicted that the victim closer to the attacker would be more likely to be compromised, since the rogue DHCP Offer would arrive first. This prediction matched the experimental results, where the victim receiving the first Offer obtained the malicious configuration.

2.3.3 Switch Comparison

Cisco Catalyst 2960 supports DHCP snooping, allowing ports to be classified as trusted or untrusted and blocking DHCP server messages from untrusted ports.

Link: www.cisco.com/c/en/us/products/switches/catalyst-2960-series-switches/index.html

The TP-Link TL-SF1005D does not support DHCP snooping, as it is an unmanaged switch without configuration capabilities or security features.

Link: www.tp-link.com/en/home-networking/soho-switch/tl-sf1005d/

2.3.4 Feasibility

The attack is feasible in networks without DHCP snooping, especially when the attacker is closer to the victim than the legitimate DHCP server. However, due to the race condition, success is not guaranteed. When DHCP snooping is enabled, the attack becomes ineffective.

2.4 ARP poisoning (MitM)

2.4.1 Network Topology

The ARP poisoning attack was performed in the network shown in Figure 4, where two hosts communicate through a switch and an attacker is connected to the same Layer 2 domain.

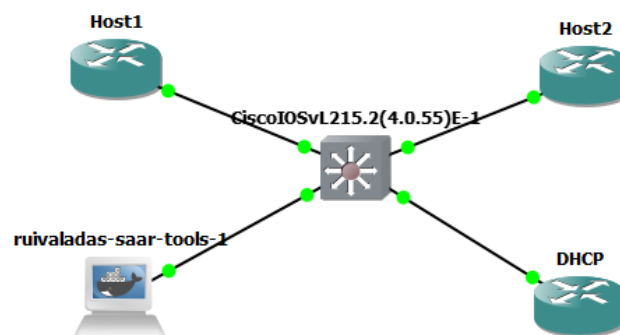


Figure 4: Network topology for ARP poisoning attack

2.4.2 Attack and Countermeasure

As observed in Wireshark (Figure 5), ARP packets indicate duplicate IP usage, confirming that the attacker is associating its MAC address with multiple IP addresses.

At the same time, ICMP traffic between the victims is still observed, which demonstrates that communication is maintained while being intercepted by the attacker, confirming a successful Man-in-the-Middle (MitM) condition.

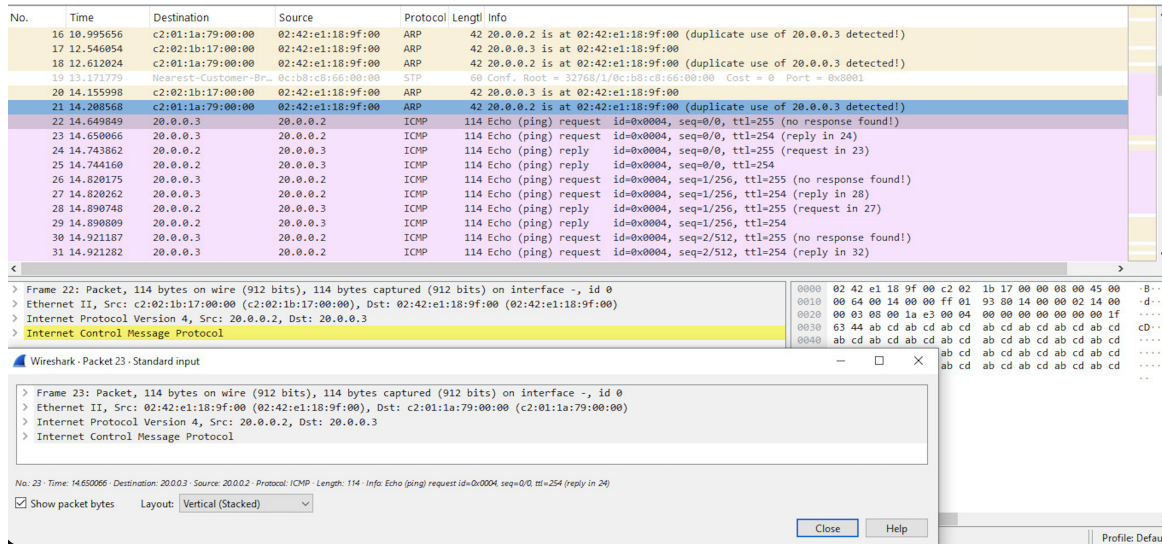


Figure 5: ARP poisoning showing duplicate IP detection and intercepted ICMP traffic

The attacker generated multiple forged ARP requests and replies, attempting to poison the ARP tables of the victims, as shown in Figure 6.

22	41.442250	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.3? Tell 20.0.0.2
24	43.473802	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.4? Tell 20.0.0.2
26	45.523090	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.3? Tell 20.0.0.2
28	47.548118	Broadcast	02:42:e1:18:9f:00	ARP	42	20.0.0.4 is at 02:42:e1:18:9f:00
29	47.597578	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.4? Tell 20.0.0.2
31	49.613918	Broadcast	02:42:e1:18:9f:00	ARP	42	20.0.0.3 is at 02:42:e1:18:9f:00
32	51.161927	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.3? Tell 20.0.0.2
34	53.178241	Broadcast	02:42:e1:18:9f:00	ARP	42	20.0.0.4 is at 02:42:e1:18:9f:00
35	53.226972	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.4? Tell 20.0.0.2
37	55.245424	Broadcast	02:42:e1:18:9f:00	ARP	42	20.0.0.3 is at 02:42:e1:18:9f:00
41	56.794383	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.3? Tell 20.0.0.2
43	58.809905	Broadcast	02:42:e1:18:9f:00	ARP	42	20.0.0.4 is at 02:42:e1:18:9f:00
44	58.862239	Broadcast	02:42:e1:18:9f:00	ARP	42	Who has 20.0.0.4? Tell 20.0.0.2

Figure 6: ARP packets generated by the attacker

However, the attack was mitigated using Dynamic ARP Inspection (DAI). The switch detected invalid ARP packets and blocked them, as shown in the system logs (Figure 7).

```
*Apr 29 10:34:18.724: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.3/10:34:18 UTC Wed Apr 29 2026])
*Apr 29 10:34:20.876: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.4/0000.000
0.0000/20.0.0.3/10:34:20 UTC Wed Apr 29 2026])
*Apr 29 10:34:20.877: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.4/10:34:20 UTC Wed Apr 29 2026])
*Apr 29 10:34:23.095: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.3/0000.000
0.0000/20.0.0.4/10:34:21 UTC Wed Apr 29 2026])
*Apr 29 10:34:24.097: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.3/10:34:23 UTC Wed Apr 29 2026])
*Apr 29 10:34:25.279: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.4/0000.000
0.0000/20.0.0.3/10:34:25 UTC Wed Apr 29 2026])
*Apr 29 10:34:25.280: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.4/10:34:25 UTC Wed Apr 29 2026])
*Apr 29 10:34:26.622: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on GigabitEthernet0/3 (not half duplex), with DHCP
FastEthernet0/0 (half duplex).
```

Figure 7: DAI log showing invalid ARP packets being denied

2.4.3 Defense Statistics

The effectiveness of DAI is confirmed by the statistics shown in Figure 8, where a high number of ARP packets were dropped by the switch.

```
Switch#show ip arp inspection statistics
```

Vlan	Forwarded	Dropped	DHCP Drops	ACL Drops
---	-----	-----	-----	-----
1	6	259	259	0

Vlan	DHCP Permits	ACL Permits	Probe Permits	Source MAC Failures
---	-----	-----	-----	-----
1	6	0	0	0

Vlan	Dest MAC Failures	IP Validation Failures	Invalid Protocol Data
---	-----	-----	-----
1	0	0	0

Figure 8: DAI statistics showing dropped ARP packets

2.4.4 AI Configuration vs Experimental Setup

The AI suggested enabling DHCP snooping and configuring Dynamic ARP Inspection on the switch, marking trusted interfaces accordingly.

This matched the experimental setup, where DHCP snooping was required for DAI to validate ARP packets. The configuration had to be adapted to the topology, particularly in defining trusted ports and VLAN configuration.

2.4.5 Switch Comparison

Cisco Catalyst 2960 supports Dynamic ARP Inspection, allowing validation of ARP packets based on DHCP snooping bindings.

Link: www.cisco.com/c/en/us/products/switches/catalyst-2960-series-switches/index.html

The TP-Link TL-SF1005D does not support DAI, as it is an unmanaged switch without security or inspection mechanisms.

Link: www.tp-link.com/en/home-networking/soho-switch/tl-sf1005d/

2.4.6 Feasibility

The attacker attempted to poison the ARP tables by sending forged ARP messages. The attack successfully created a Man-in-the-Middle condition, as observed in Wireshark. However, based on the switch logs and DAI statistics, these packets were detected and blocked.

Therefore, although the attack is feasible in networks without protection, in this setup it was not successful due to the correct configuration of Dynamic ARP Inspection.

2.5 DNS spoofing

2.6 RIP Poisoning

2.6.1 Network Topology

The RIP poisoning attack was performed using the topology shown in Figure 9. The Web browser, the legitimate Web server and the fake Web server are located in different subnets, with the attacker collocated with the fake Web server.

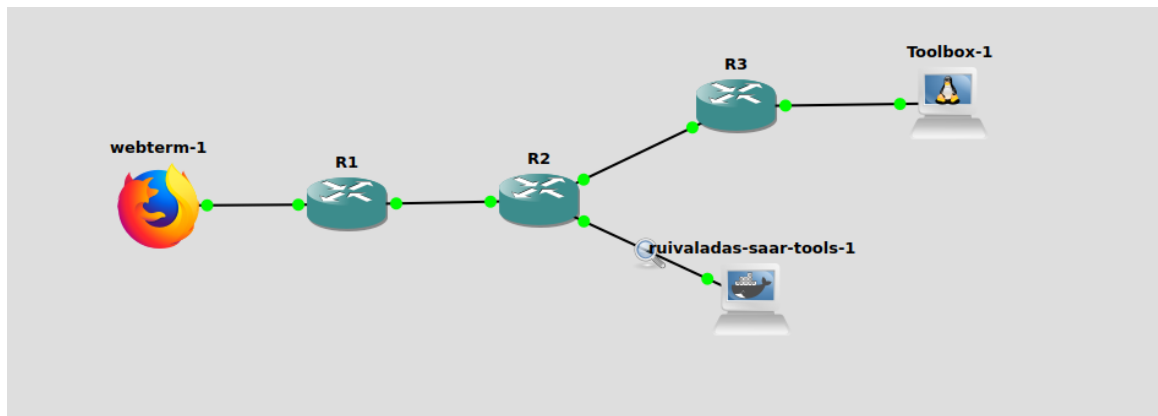


Figure 9: Network topology used for the RIP poisoning attack

2.6.2 Attack and Countermeasure

The attacker sent forged RIPv2 Response messages in order to poison the routing tables of the routers. These messages advertised a route to the legitimate Web server network through the attacker, causing traffic from the browser to be redirected to the fake Web server.

As shown in Figure 10, Wireshark captured multiple RIPv2 Response packets sent to the multicast address 224.0.0.9, which is used by RIPv2 routers. The captured packet advertises the network 10.0.0.0 with metric 5 and subnet mask 255.255.255.128, showing that the attacker injected routing information into the network.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	c2:02:37:fd:00:10	c2:02:37:fd:00:10	LOOP	60	Reply
2	1.733278	172.16.0.2	224.0.0.9	RIPv2	66	Response
3	3.737119	172.16.0.2	224.0.0.9	RIPv2	66	Response
4	5.740474	172.16.0.2	224.0.0.9	RIPv2	66	Response
5	6.658340	192.168.1.2	10.0.0.2	TCP	74	54068 → 80 [SYN] Seq=0 Win=6424
6	6.658522	10.0.0.2	192.168.1.2	TCP	74	80 → 54068 [SYN, ACK] Seq=0 Ack=
7	6.688746	192.168.1.2	10.0.0.2	TCP	66	54068 → 80 [ACK] Seq=1 Ack=1 Wi
8	6.698864	192.168.1.2	10.0.0.2	HTTP	476	GET / HTTP/1.1
9	6.699201	10.0.0.2	192.168.1.2	TCP	66	80 → 54068 [ACK] Seq=1 Ack=411
10	6.699222	10.0.0.2	192.168.1.2	HTTP	254	HTTP/1.1 304 Not Modified
11	6.739220	192.168.1.2	10.0.0.2	TCP	66	54068 → 80 [ACK] Seq=411 Ack=18
12	7.744338	172.16.0.2	224.0.0.9	RIPv2	66	Response
13	9.748365	172.16.0.2	224.0.0.9	RIPv2	66	Response
14	9.991713	c2:02:37:fd:00:10	c2:02:37:fd:00:10	LOOP	60	Reply
15	11.746292	02:42:b2:9b:6c:00	c2:02:37:fd:00:10	ARP	42	Who has 172.16.0.1? Tell 172.16
16	11.751331	172.16.0.2	224.0.0.9	RIPv2	66	Response
17	11.752374	c2:02:37:fd:00:10	02:42:b2:9b:6c:00	ARP	60	172.16.0.1 is at c2:02:37:fd:00
18	13.755114	172.16.0.2	224.0.0.9	RIPv2	66	Response
19	15.758195	172.16.0.2	224.0.0.9	RIPv2	66	Response
20	15.898924	172.16.0.1	224.0.0.9	RIPv2	126	Response
21	16.875705	192.168.1.2	10.0.0.2	TCP	66	[TCP Keep-Alive] 54068 → 80 [AC
22	16.875934	10.0.0.2	192.168.1.2	TCP	66	[TCP Keep-Alive ACK] 80 → 54068
23	17.760482	172.16.0.2	224.0.0.9	RIPv2	66	Response
24	17.762445	172.16.0.2	224.0.0.9	RIPv2	66	Response

Source Port: 520
Destination Port: 520
Length: 32
Checksum: 0x63f8 [unverified]
[Checksum Status: Unverified]
[Stream index: 0]
▸ [Timestamps]
UDP payload (24 bytes)
▾ Routing Information Protocol
Command: Response (2)
Version: RIPv2 (2)
▾ IP Address: 10.0.0.0, Metric: 5
Address Family: IP (2)
Route Tag: 0
IP Address: 10.0.0.0
Netmask: 255.255.255.128
Next Hop: 0.0.0.0
Metric: 5

Figure 10: Forged RIPv2 response advertising a route to the target network

The effect of the attack was confirmed in the browser. As shown in Figure 11, the Web browser was redirected to the fake Web server, displaying the message “FAKE SERVER - ATTACK SUCCESS”.

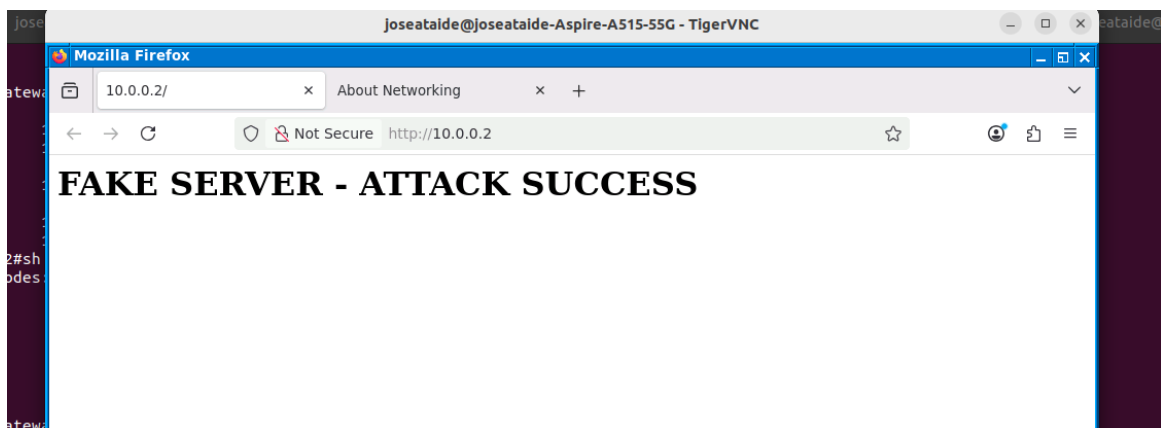


Figure 11: Browser redirected to the fake Web server

Additional Wireshark captures show TCP and HTTP communication between the browser and the fake server, confirming that the traffic was redirected after the routing tables were poisoned.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	c2:02:37:fd:00:10	c2:02:37:fd:00:10	LOOP	60	Reply
2	1.733278	172.16.0.2	224.0.0.9	RIPv2	66	Response
3	3.737119	172.16.0.2	224.0.0.9	RIPv2	66	Response
4	5.748474	172.16.0.2	224.0.0.9	RIPv2	66	Response
5	6.658340	192.168.1.2	10.0.0.2	TCP	74	54068 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=1606159760 TSecr=
6	6.658522	10.0.0.2	192.168.1.2	TCP	74	80 → 54068 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM=1 TSval=160606
7	6.688746	192.168.1.2	10.0.0.2	TCP	66	54068 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1606159785 TSecr=1606068852
8	6.698864	192.168.1.2	10.0.0.2	HTTP	476	GET / HTTP/1.1
9	6.699281	10.0.0.2	192.168.1.2	TCP	66	80 → 54068 [ACK] Seq=1 Ack=411 Win=64768 Len=0 TSval=1606068892 TSecr=1606159785
10	6.699222	10.0.0.2	192.168.1.2	HTTP	254	HTTP/1.1 304 Not Modified
11	6.739220	192.168.1.2	10.0.0.2	TCP	66	54068 → 80 [ACK] Seq=411 Ack=189 Win=64128 Len=0 TSval=1606159835 TSecr=1606068892
12	7.744338	172.16.0.2	224.0.0.9	RIPv2	66	Response
13	9.748365	172.16.0.2	224.0.0.9	RIPv2	66	Response
14	9.991713	c2:02:37:fd:00:10	c2:02:37:fd:00:10	LOOP	60	Reply
15	11.746292	02:42:b2:9b:6c:00	c2:02:37:fd:00:10	ARP	42	Who has 172.16.0.1? Tell 172.16.0.2
16	11.751331	172.16.0.2	224.0.0.9	RIPv2	66	Response
17	11.752374	c2:02:37:fd:00:10	02:42:b2:9b:6c:00	ARP	60	172.16.0.1 is at c2:02:37:fd:00:10
18	13.755114	172.16.0.2	224.0.0.9	RIPv2	66	Response
19	15.768195	172.16.0.2	224.0.0.9	RIPv2	66	Response
20	15.898924	172.16.0.1	224.0.0.9	RIPv2	126	Response
21	16.875795	192.168.1.2	10.0.0.2	TCP	66	[TCP Keep-Alive] 54068 → 80 [ACK] Seq=410 Ack=189 Win=64128 Len=0 TSval=1606169974
22	16.875934	10.0.0.2	192.168.1.2	TCP	66	[TCP Keep-Alive ACK] 80 → 54068 [ACK] Seq=189 Ack=411 Win=64768 Len=0 TSval=16060615
23	17.760482	172.16.0.2	224.0.0.9	RIPv2	66	Response
24	19.763415	172.16.0.2	224.0.0.9	RIPv2	66	Response
25	19.997215	c2:02:37:fd:00:10	c2:02:37:fd:00:10	LOOP	60	Reply

Figure 12: TCP and HTTP traffic redirected to the fake Web server

As a countermeasure, RIPv2 authentication can be enabled on the routers. With authentication, routers only accept RIP updates from trusted devices, preventing forged routing updates from being installed in the routing table.

2.6.3 Effectiveness of the Attack

The success of the attack depends on the relative location of the browser, legitimate Web server and fake Web server. Since the attacker is placed near the fake Web server, it can advertise a route that makes the fake server appear as a better path to the target network.

The prefix length advertised by the attacker is also important. A more specific prefix is preferred due to the longest prefix match rule. Therefore, if the attacker advertises a more specific route than the legitimate one, the routers may select the malicious route.

RIP auto-summary can reduce the effectiveness of the attack because it summarizes routes at classful network boundaries. When auto-summary is disabled, more specific routes can be advertised, making the attack more effective.

Figures 13 and 14 show repeated RIPv2 Response messages, confirming that the attacker continuously injected routing updates into the network.

No.	Time	Source	Destination	Protocol	Length	Info
5	4.711615	192.168.23.2	224.0.0.9	RIPv2	106	Response
8	13.173999	192.168.23.2	224.0.0.9	RIPv2	66	Response
13	34.374262	192.168.23.2	224.0.0.9	RIPv2	106	Response
16	42.556733	192.168.23.2	224.0.0.9	RIPv2	66	Response
23	63.616773	192.168.23.2	224.0.0.9	RIPv2	106	Response
24	68.429552	192.168.23.2	224.0.0.9	RIPv2	66	Response

Figure 13: Filtered RIPv2 packets sent to multicast address 224.0.0.9

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
2	0.268414	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
3	3.282191	c2:02:37:fd:00:00	CDP/VTP/DTP/PAgP/UD...	CDP	349	Device ID: R2 Port ID: FastEthernet0/0
4	3.481026	c2:03:38:22:00:00	CDP/VTP/DTP/PAgP/UD...	CDP	349	Device ID: R3 Port ID: FastEthernet0/0
5	4.711615	192.168.23.2	224.0.0.9	RIPv2	106	Response
6	9.998962	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
7	10.275370	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
8	13.173999	192.168.23.3	224.0.0.9	RIPv2	66	Response
9	20.005237	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
10	20.270487	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
11	30.003637	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
12	30.270107	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
13	34.374262	192.168.23.2	224.0.0.9	RIPv2	106	Response
14	40.001557	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
15	40.272070	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
16	42.556733	192.168.23.3	224.0.0.9	RIPv2	66	Response
17	49.995916	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
18	50.267336	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
19	60.000765	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
20	60.272327	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
21	63.284522	c2:02:37:fd:00:00	CDP/VTP/DTP/PAgP/UD...	CDP	349	Device ID: R2 Port ID: FastEthernet0/0
22	63.475356	c2:03:38:22:00:00	CDP/VTP/DTP/PAgP/UD...	CDP	349	Device ID: R3 Port ID: FastEthernet0/0
23	63.616773	192.168.23.2	224.0.0.9	RIPv2	106	Response
24	68.429552	192.168.23.3	224.0.0.9	RIPv2	66	Response
25	69.998044	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
26	70.272005	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
27	79.995333	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
28	80.267995	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply

Figure 14: Repeated RIPv2 responses observed during the attack

2.6.4 AI Prediction vs Experimental Results

The AI predicted that the attacker would be more successful when advertising a more specific prefix, because routers prefer the longest prefix match. It also predicted that disabling RIP auto-summary would make the attack more effective, since the malicious route would not be summarized.

This prediction matched the experimental behavior. The captured RIP packets show that the attacker advertised specific routing information, and the browser traffic was redirected to the fake Web server. Therefore, the observed results confirm that prefix length and auto-summary directly influence route selection.

2.6.5 Feasibility

This attack is feasible in networks using RIP without authentication. Since RIP accepts routing updates from neighboring devices, an attacker can inject forged routes and redirect traffic.

However, in modern networks, RIP is less common and authentication can prevent this attack. Therefore, the attack is mainly feasible in poorly configured or legacy networks.

3 Firewalls

3.1 Classical firewalls versus Zone-Based Policy Firewalls

3.1.1 Firewall Validation

The firewall must enforce the following rules:

1. Block all traffic from INSIDE to OUTSIDE;
2. Allow only ICMP and HTTP traffic from OUTSIDE to the server;
3. Deny access to the firewall interfaces.

3.1.2 Classical Firewall (CBAC)

In the classical firewall, traffic filtering is based on ACLs combined with stateful inspection rules. As shown in the slides, inspection rules only track sessions and do not impose restrictions by themselves.

Initially, the configuration allowed access to multiple server ports because the ACL was too permissive. This confirms that **inspect rules do not have an implicit deny**, meaning that unwanted traffic may still pass if not explicitly blocked.

After correcting the ACLs, the observed behavior was:

1. Only HTTP and ICMP traffic to the server was allowed;
2. All other ports were classified as filtered by NMAP;
3. No traffic from INSIDE to OUTSIDE was possible due to explicit deny rules.

3.1.3 ZBPF

In ZBPF, interfaces are assigned to zones and policies are applied between zones. By default, traffic between different zones is dropped unless a policy is defined.

After configuration, the firewall behavior was:

1. Only explicitly permitted traffic (HTTP and ICMP to the server) was allowed;
2. All other inter-zone traffic was blocked by default;
3. Access to firewall interfaces was denied (controlled via the self zone).

Therefore, both firewalls satisfy the specification, but ZBPF enforces it more naturally due to its default deny model.

3.2 NMAP Operation

NMAP is used to discover hosts and scan ports by actively probing the network.

3.2.1 Host Discovery

When using `nmap -sn`, NMAP determines if hosts are active by sending:

1. ICMP Echo requests;
2. TCP SYN packets to common ports (e.g., port 80);
3. ARP requests in local networks.

A host is considered active if it replies to any of these probes.

3.2.2 Port Scanning

For port scanning, NMAP typically uses a TCP SYN scan:

1. SYN-ACK response $\Rightarrow$ port is open;
2. RST response $\Rightarrow$ port is closed;
3. No response or ICMP error $\Rightarrow$ port is filtered.

This method is efficient because it does not complete the TCP connection, allowing fast scanning of multiple ports.

3.3 CBAC vs ZBPF

The main differences between CBAC and ZBPF are:

1. **Configuration model:** CBAC is interface-based and relies on ACLs plus inspection rules, while ZBPF is zone-based and uses zone-pairs, class-maps, and policy-maps.
2. **Default behavior:** In CBAC, traffic may be allowed unless explicitly denied by ACLs. In ZBPF, traffic between zones is denied by default unless a policy allows it.
3. **Stateful inspection:** Both support stateful inspection, but in CBAC it depends on inspect rules applied to interfaces, whereas in ZBPF it is integrated into policy actions (inspect, pass, drop).
4. **Security and control:** ZBPF provides stricter and more modular control, since all inter-zone traffic must be explicitly defined. CBAC is less structured and more prone to configuration errors.

In conclusion, although both approaches can enforce the required policy, ZBPF offers a more robust, scalable, and secure solution.

3.4 Protecting a Campus Network using ZBPF

3.4.1 Network Topology

The network is divided into four security zones: PR1, PR2, DMZ, and OUT, as shown in Figure 15.

The firewall is implemented on a Cisco 7200 router using Zone-Based Policy Firewall (ZBPF). The DMZ contains Web, Email, and DNS servers, while PR1 and PR2 represent private networks.

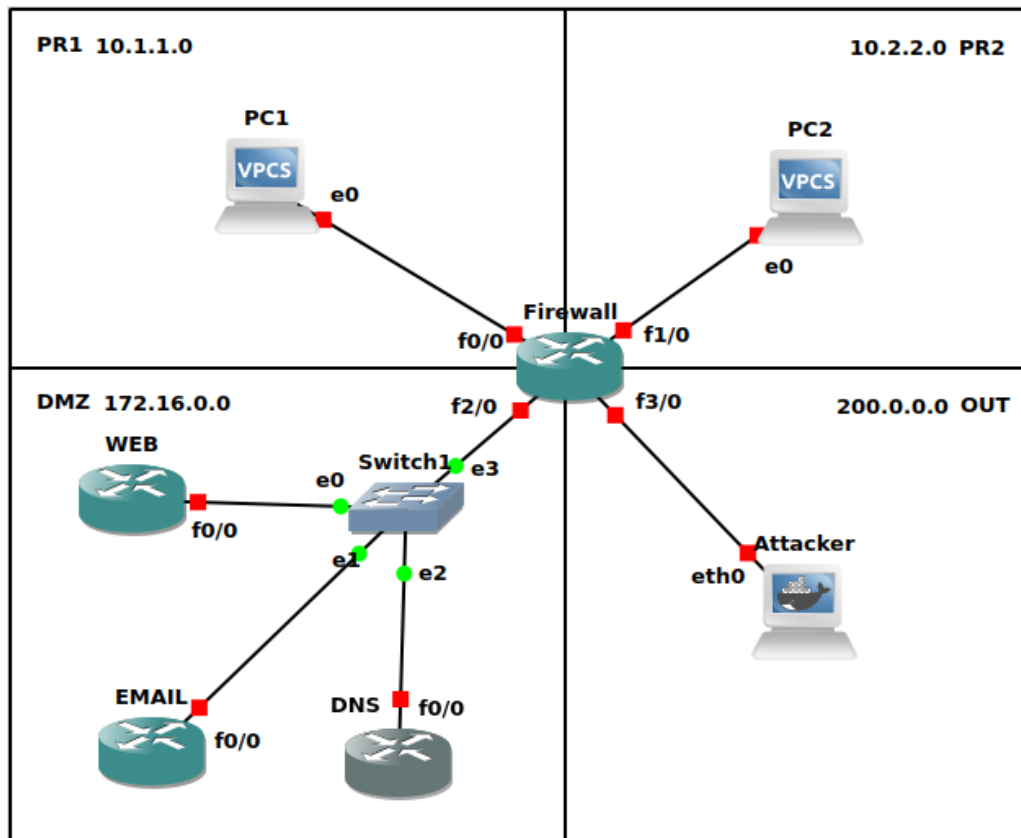


Figure 15: Campus network protected by a ZBPF firewall

3.4.2 Firewall Configuration

Four security zones were created and associated with the firewall interfaces. By default, communication between zones is denied unless explicitly permitted.

NAT overload was configured to allow private hosts to access the OUT zone while hiding internal IP addresses.

Traffic policies were implemented using ACLs, class-maps, policy-maps, and zone-pairs. Only the required services were allowed:

- HTTP/HTTPS to the Web server;
- SMTP to the Email server;

- DNS queries to the DNS server.

No zone-pairs were configured between PR1 and PR2, nor from OUT towards the private zones, resulting in implicit traffic denial.

3.4.3 Testing and Validation

The firewall policies were validated using NMAP and Wireshark.

PR1 to PR2 Communication between private zones was blocked. NMAP scans returned no responses, confirming inter-zone isolation.

No.	Time	Source	Destination	Protocol	Length	Info
6	29.102670	10.1.1.10	10.2.2.10	TCP	58	46207 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
7	29.102682	10.1.1.10	10.2.2.10	TCP	54	46207 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0
8	29.102689	10.1.1.10	10.2.2.10	ICMP	54	Timestamp request id=0x4e6a, seq=0/0, ttl=38
9	29.999990	ca:01:66:fc:00:00	ca:01:66:fc:00:00	LOOP	60	Reply
10	31.104125	10.1.1.10	10.2.2.10	ICMP	54	Timestamp request id=0x4f7c, seq=0/0, ttl=43
11	31.104177	10.1.1.10	10.2.2.10	TCP	54	46208 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0
12	31.104187	10.1.1.10	10.2.2.10	TCP	58	46208 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	31.104195	10.1.1.10	10.2.2.10	ICMP	42	Echo (ping) request id=0x0bd9, seq=0/0, ttl=59 (no response found!)
14	34.292065	02:42:f9:55:ef:00	ca:01:66:fc:00:00	ARP	42	Who has 10.1.1.254? Tell 10.1.1.10
15	34.298669	ca:01:66:fc:00:00	02:42:f9:55:ef:00	ARP	60	10.1.1.254 is at ca:01:66:fc:00:00
16	39.993986	ca:01:66:fc:00:00	ca:01:66:fc:00:00	LOOP	60	Reply
17	50.000384	ca:01:66:fc:00:00	ca:01:66:fc:00:00	LOOP	60	Reply
18	52.952688	10.1.1.10	10.2.2.10	TCP	58	35631 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	52.952735	10.1.1.10	10.2.2.10	TCP	58	35631 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
20	52.952747	10.1.1.10	10.2.2.10	TCP	58	35631 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
21	52.952754	10.1.1.10	10.2.2.10	TCP	58	35631 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	52.952764	10.1.1.10	10.2.2.10	TCP	58	35631 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	52.952774	10.1.1.10	10.2.2.10	TCP	58	35631 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
24	52.952782	10.1.1.10	10.2.2.10	TCP	58	35631 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
25	52.952794	10.1.1.10	10.2.2.10	TCP	58	35631 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
26	52.952806	10.1.1.10	10.2.2.10	TCP	58	35631 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
27	52.952814	10.1.1.10	10.2.2.10	TCP	58	35631 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
28	54.954112	10.1.1.10	10.2.2.10	TCP	58	35632 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
29	54.954156	10.1.1.10	10.2.2.10	TCP	58	35632 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	54.954168	10.1.1.10	10.2.2.10	TCP	58	35632 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
31	54.954175	10.1.1.10	10.2.2.10	TCP	58	35632 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	54.954186	10.1.1.10	10.2.2.10	TCP	58	35632 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
33	54.954196	10.1.1.10	10.2.2.10	TCP	58	35632 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
34	54.954204	10.1.1.10	10.2.2.10	TCP	58	35632 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	54.954215	10.1.1.10	10.2.2.10	TCP	58	35632 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
36	54.954223	10.1.1.10	10.2.2.10	TCP	58	35632 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	54.954231	10.1.1.10	10.2.2.10	TCP	58	35632 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	55.955116	10.1.1.10	10.2.2.10	TCP	58	35631 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	55.955166	10.1.1.10	10.2.2.10	TCP	58	35631 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
40	55.955175	10.1.1.10	10.2.2.10	TCP	58	35631 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
41	55.955183	10.1.1.10	10.2.2.10	TCP	58	35631 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
42	55.955193	10.1.1.10	10.2.2.10	TCP	58	35631 → 139 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
43	55.955202	10.1.1.10	10.2.2.10	TCP	58	35631 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
44	55.955211	10.1.1.10	10.2.2.10	TCP	58	35631 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
45	55.955220	10.1.1.10	10.2.2.10	TCP	58	35631 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
46	55.955228	10.1.1.10	10.2.2.10	TCP	58	35631 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	55.955240	10.1.1.10	10.2.2.10	TCP	58	35631 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
48	55.955251	10.1.1.10	10.2.2.10	TCP	58	35632 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
49	55.955262	10.1.1.10	10.2.2.10	TCP	58	35632 → 5000 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Figure 16: NMAP scan from PR1 to PR2 showing no replies, confirming inter-zone isolation enforced by the firewall

PR1 to OUT NAT operation was validated by scanning external hosts. Wireshark captures confirmed source address translation using the firewall public IP.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ca:01:66:fc:00:54	ca:01:66:fc:00:54	LOOP	60	Reply
2	7.165880	200.0.0.254	200.0.0.10	ICMP	42	Echo (ping) request id=0xcf43, seq=0/0, ttl=48 (reply in 5)
3	7.165966	200.0.0.254	200.0.0.10	TCP	58	47606 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
4	7.165980	200.0.0.254	200.0.0.10	ICMP	54	Timestamp request id=0xda93, seq=0/0, ttl=45
5	7.166107	200.0.0.10	200.0.0.254	ICMP	42	Echo (ping) reply id=0xcf43, seq=0/0, ttl=64 (request in 2)
6	7.166130	200.0.0.10	200.0.0.254	TCP	54	443 → 47606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	7.166140	200.0.0.10	200.0.0.254	ICMP	54	Timestamp reply id=0xda93, seq=0/0, ttl=64
8	7.206274	200.0.0.254	200.0.0.10	TCP	58	47862 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
9	7.206480	200.0.0.10	200.0.0.254	TCP	54	80 → 47862 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	8.407064	200.0.0.254	200.0.0.10	TCP	58	47862 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	8.407292	200.0.0.10	200.0.0.254	TCP	54	443 → 47862 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12	9.705331	200.0.0.254	200.0.0.10	TCP	58	47873 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	9.705535	200.0.0.10	200.0.0.254	TCP	54	80 → 47873 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14	9.997318	ca:01:66:fc:00:54	ca:01:66:fc:00:54	LOOP	60	Reply
15	11.013732	200.0.0.254	200.0.0.10	TCP	58	47874 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
16	11.013933	200.0.0.10	200.0.0.254	TCP	54	80 → 47874 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17	12.414772	02:42:51:a1:df:00	ca:01:66:fc:00:54	ARP	42	Who has 200.0.0.254? Tell 200.0.0.10
18	12.422328	ca:01:66:fc:00:54	02:42:51:a1:df:00	ARP	60	200.0.0.254 is at ca:01:66:fc:00:54
19	12.663798	ca:01:66:fc:00:54	CDP/VTP/DTP/PagP/UD...	CDP	374	Device ID: Firewall Port ID: FastEthernet3/0

Figure 17: Traffic from PR1 to the OUT zone showing NAT translation and access only to permitted services

PR1 to DMZ Only HTTP and HTTPS services were reachable on the DMZ Web server, while all other ports were filtered.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
2	8.592493	c2:03:17:f5:00:00	CDP/VTP/DTP/PagP/UD...	CDP	352	Device ID: EMAIL Port ID: FastEthernet0/0
3	8.613302	c2:02:17:d5:00:00	CDP/VTP/DTP/PagP/UD...	CDP	350	Device ID: WEB Port ID: FastEthernet0/0
4	8.854406	c2:04:18:14:00:00	CDP/VTP/DTP/PagP/UD...	CDP	350	Device ID: DNS Port ID: FastEthernet0/0
5	10.001566	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
6	18.007292	ca:01:66:fc:00:38	CDP/VTP/DTP/PagP/UD...	CDP	374	Device ID: Firewall Port ID: FastEthernet2/0
7	19.999330	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
8	20.926049	10.1.1.10	172.16.0.10	ICMP	42	Echo (ping) request id=0xfb72, seq=0/0, ttl=38 (reply in 11)
9	20.926118	10.1.1.10	172.16.0.10	TCP	58	63664 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	20.926136	10.1.1.10	172.16.0.10	ICMP	54	Timestamp request id=0xfc7c, seq=0/0, ttl=55
11	20.933514	172.16.0.10	10.1.1.10	ICMP	60	Echo (ping) reply id=0xfb72, seq=0/0, ttl=255 (request in 8)
12	20.943595	172.16.0.10	10.1.1.10	TCP	60	443 → 63664 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
13	20.953642	172.16.0.10	10.1.1.10	ICMP	60	Timestamp reply id=0xfc7c, seq=0/0, ttl=255
14	20.956320	10.1.1.10	172.16.0.10	TCP	54	63664 → 443 [RST] Seq=1 Win=0 Len=0
15	20.966448	10.1.1.10	172.16.0.10	TCP	58	63920 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
16	20.973835	172.16.0.10	10.1.1.10	TCP	60	443 → 63920 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
17	20.986654	10.1.1.10	172.16.0.10	TCP	54	63920 → 443 [RST] Seq=1 Win=0 Len=0
18	20.986742	10.1.1.10	172.16.0.10	TCP	58	63920 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	20.994006	172.16.0.10	10.1.1.10	TCP	60	80 → 63920 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
20	21.006788	10.1.1.10	172.16.0.10	TCP	54	63920 → 80 [RST] Seq=1 Win=0 Len=0
21	22.274638	10.1.1.10	172.16.0.10	TCP	58	63931 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	22.281677	172.16.0.10	10.1.1.10	TCP	60	443 → 63931 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
23	22.294798	10.1.1.10	172.16.0.10	TCP	54	63931 → 443 [RST] Seq=1 Win=0 Len=0
24	23.572878	10.1.1.10	172.16.0.10	TCP	58	63932 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
25	23.580159	172.16.0.10	10.1.1.10	TCP	60	443 → 63932 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
26	23.593028	10.1.1.10	172.16.0.10	TCP	54	63932 → 443 [RST] Seq=1 Win=0 Len=0
27	24.879034	10.1.1.10	172.16.0.10	TCP	58	63933 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
28	24.888499	172.16.0.10	10.1.1.10	TCP	60	443 → 63933 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
29	24.899209	10.1.1.10	172.16.0.10	TCP	54	63933 → 443 [RST] Seq=1 Win=0 Len=0
30	30.001393	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply

Figure 18: NMAP scan from PR1 to the DMZ showing only HTTP and HTTPS services accessible

OUT to Private Zones Connections from the OUT zone to PR1 and PR2 were unsuccessful, confirming that private networks are protected from external access.

OUT to DMZ Only the permitted public services were accessible from the OUT zone.

Apply a display filter ... <Ctrl-/>					
No.	Time	Source	Destination	Protocol	Length Info
1	0.000000	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60 Reply
2	10.000576	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60 Reply
3	14.012546	200.0.0.10	172.16.0.10	ICMP	42 Echo (ping) request id=0x996e, seq=0/0, ttl=40 (reply in 5)
4	14.012618	200.0.0.10	172.16.0.10	TCP	58 49731 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	14.019800	172.16.0.10	200.0.0.10	ICMP	60 Echo (ping) reply id=0x996e, seq=0/0, ttl=255 (request in 3)
6	14.022644	200.0.0.10	172.16.0.10	ICMP	54 Timestamp request id=0xc502, seq=0/0, ttl=56
7	14.029861	172.16.0.10	200.0.0.10	TCP	60 443 → 49731 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
8	14.040520	172.16.0.10	200.0.0.10	ICMP	60 Timestamp reply id=0xc502, seq=0/0, ttl=255
9	14.042790	200.0.0.10	172.16.0.10	TCP	54 49731 → 443 [RST] Seq=1 Win=0 Len=0
10	14.062904	200.0.0.10	172.16.0.10	TCP	58 49987 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	14.076271	172.16.0.10	200.0.0.10	TCP	60 80 → 49987 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
12	14.083105	200.0.0.10	172.16.0.10	TCP	54 49987 → 80 [RST] Seq=1 Win=0 Len=0
13	15.360714	200.0.0.10	172.16.0.10	TCP	58 49998 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	15.368205	172.16.0.10	200.0.0.10	TCP	60 80 → 49998 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
15	15.380916	200.0.0.10	172.16.0.10	TCP	54 49998 → 80 [RST] Seq=1 Win=0 Len=0
16	15.380968	200.0.0.10	172.16.0.10	TCP	58 49987 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
17	15.388361	172.16.0.10	200.0.0.10	TCP	60 443 → 49987 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
18	15.401058	200.0.0.10	172.16.0.10	TCP	54 49987 → 443 [RST] Seq=1 Win=0 Len=0
19	16.658930	200.0.0.10	172.16.0.10	TCP	58 49999 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
20	16.667333	172.16.0.10	200.0.0.10	TCP	60 80 → 49999 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
21	16.679106	200.0.0.10	172.16.0.10	TCP	54 49999 → 80 [RST] Seq=1 Win=0 Len=0
22	17.926681	200.0.0.10	172.16.0.10	TCP	58 50000 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	17.934930	172.16.0.10	200.0.0.10	TCP	60 80 → 50000 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
24	17.946846	200.0.0.10	172.16.0.10	TCP	54 50000 → 80 [RST] Seq=1 Win=0 Len=0
25	18.592120	c2:03:17:f5:00:00	CDP/VTP/DTP/PagP/UD...	CDP	352 Device ID: EMAIL Port ID: FastEthernet0/0
26	18.619041	c2:02:17:d5:00:00	CDP/VTP/DTP/PagP/UD...	CDP	350 Device ID: WEB Port ID: FastEthernet0/0
27	18.854289	c2:04:18:14:00:00	CDP/VTP/DTP/PagP/UD...	CDP	350 Device ID: DNS Port ID: FastEthernet0/0
28	20.009530	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60 Reply

Figure 19: Access from the OUT zone to the DMZ restricted to publicly allowed services

Firewall Management SSH access to the firewall was successful after enabling compatibility options for legacy Cisco IOS cryptographic algorithms. Telnet access was blocked, confirming secure management access.

3.4.4 Conclusion

The experimental results confirm that the ZBPF firewall correctly enforces inter-zone isolation, controlled access to DMZ services, NAT operation, and secure firewall management.

3.5 Defense against DoS attacks

4 AAA

4.1 AAA with TACACS+

4.1.1 AAA Configuration

AAA was enabled on the router using the IOS `aaa new-model` command. A local administrative user was initially configured to guarantee fallback access in case the TACACS+ server became unavailable.

```
username admin privilege 15 password 0 admin
enable password cisco
aaa new-model
```

The TACACS+ server was then configured on the router:

```
tacacs server Server-T
address ipv4 10.0.0.1
key mysecret123
```

Authentication, authorization, and accounting policies were configured as follows:

```
aaa authentication login default group tacacs+ local
aaa authorization exec default group tacacs+ local
aaa authorization commands 15 default group tacacs+ local
aaa accounting exec default start-stop group tacacs+
```

Remote access through Telnet was enabled on the VTY lines:

```
line vty 0 4
login authentication default
transport input telnet
```

On the TACACS+ server, users and permissions were configured in the file:

```
/etc/tacacs+/tac_plus.conf
```

The administrative user was configured with privilege level 15:

```
user = gns3 {  
    login = des HASH  
  
    service = exec {  
        priv-lvl = 15  
    }  
  
    cmd = .* {  
        permit .*  
    }  
}
```

A restricted user was also configured with limited command authorization:

```
user = readonly {  
    login = cleartext "gns3"  
  
    service = exec {  
        priv-lvl = 1  
    }  
  
    cmd = show {  
        permit .*  
    }  
}
```

4.1.2 Authentication, Authorization and Accounting

AAA separates administrative access control into three distinct functions.

Authentication Authentication was implemented using TACACS+ server-based login validation. When a Telnet session was initiated, R1 forwarded the supplied username and password to the TACACS+ server for verification.

Authentication was validated by successfully logging into the router using the centralized TACACS+ user accounts.

```
root@TelnetClient:~# telnet 222.10.10.254  
  
Username: gns3  
Password:  
  
R1#
```

Authorization Authorization policies were configured on the TACACS+ server to provide different privilege levels to different users.

The user `gns3` received privilege level 15 and full administrative access, while the `readonly` user was restricted to monitoring commands only.

Authorization was validated by attempting privileged configuration commands using the restricted account.

```
R1>enable
Password:

R1#configure terminal
Command authorization failed
```

Accounting Accounting was configured to log TACACS+ administrative activity on the AAA server.

Session information and executed administrative actions were recorded in:

```
/var/log/tac_plus.acct
```

The generated logs include usernames, timestamps, session start and stop events, and command execution information.

```
# Created by Henry-Nicolas Tourneur
# See man(5) tac_plus.conf for more details

accounting file = /var/log/tac_plus.acct

key = mysecret123

user = DEFAULT {
    login = PAM
    service = ppp protocol = ip {}
}

user = gns3 {
    login = des 0dmbjUvhfS0jk

    service = exec {
        priv-lvl = 15
    }

    cmd = .* {
        permit .*
    }
}

user = readonly {
    login = cleartext "gns3"

    service = exec {
        priv-lvl = 1
    }

    cmd = show {
```

```
    permit .*
  }
}

group = admin {
  default service = permit

  service = exec {
    priv-lvl = 15
  }
}

group = read-only {
  service = exec {
    priv-lvl = 15
  }

  cmd = show {
    permit .*
  }

  cmd = write {
    permit term
  }

  cmd = dir {
    permit .*
  }

  cmd = admin {
    permit .*
  }

  cmd = terminal {
    permit .*
  }

  cmd = more {
    permit .*
  }

  cmd = exit {
    permit .*
  }

  cmd = logout {
    permit .*
  }
}
```

4.1.3 Password Configuration

Two different types of passwords were configured during the exercise.

The first was the TACACS+ shared secret, used to encrypt communication between the router and the TACACS+ server:

```
key = mysecret123
```

This same key was configured on both the router and the AAA server.

The second type corresponds to user authentication passwords. Initially, cleartext passwords were used, but later DES-encrypted hashes were configured.


```
tac_pwd
```

The generated hash was then inserted into the TACACS+ configuration:

```
login = des HASH
```

Authentication tests confirmed that users could still successfully log into the router using the original plaintext password, while the TACACS+ server stored only the DES hash.

4.1.4 Wireshark Analysis

Wireshark was used to capture and analyze TACACS+ traffic exchanged between the router and the AAA server.

TACACS+ communication uses TCP port 49. The packet captures show the establishment of TCP sessions followed by TACACS+ authentication, authorization, and accounting exchanges.

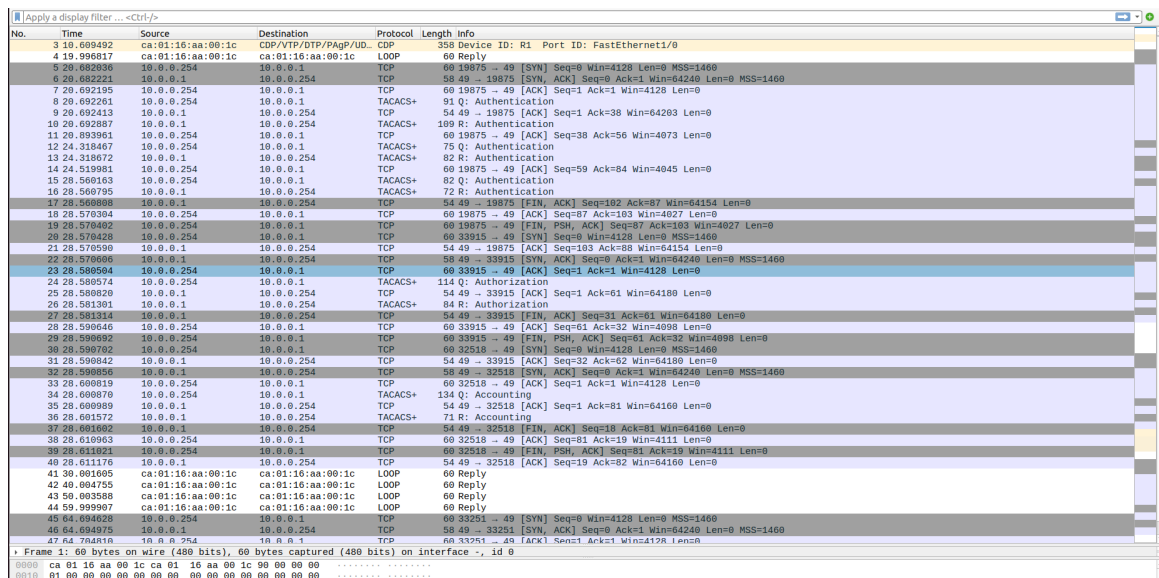


Figure 20: Wireshark capture of TACACS+ authentication traffic

Authentication packets were generated when users attempted to log into the router. Authorization packets were exchanged when IOS requested permission for command execution and privilege assignment. Accounting packets were generated when user sessions started or ended.

Because TACACS+ encrypts packet payloads, Wireshark decryption required the shared secret configured between the router and the TACACS+ server.

4.1.5 Benefits of Server-Based AAA

Compared to local AAA, server-based AAA provides centralized management of user accounts and permissions across multiple devices.

This approach simplifies administration, improves scalability, and enables detailed auditing of administrative activity. TACACS+ also provides granular command authorization, allowing different users to receive different privilege levels and command permissions.

Overall, centralized AAA provides significantly better security, control, and accountability than locally configured authentication alone.

4.2 AAA with Radius

4.3 802.1X Authentication

IEEE 802.1X authentication was implemented using a Cisco IOSvL2 switch as the authenticator, a Linux host running `wpa_supplicant` as the supplicant, and a FreeRADIUS server as the authentication server.

The switch ports were configured for 802.1X port-based authentication using RADIUS.

```
aaa new-model
aaa authentication dot1x default group radius

radius server AAAServer
address ipv4 10.1.1.50 auth-port 1812 acct-port 1813
key gns3

dot1x system-auth-control
```

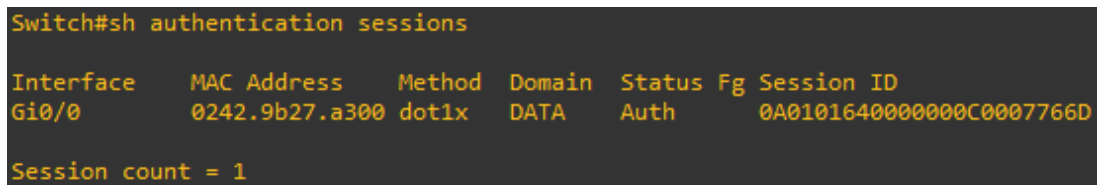
The supplicant configuration used PEAP with MSCHAPv2 authentication:

```
network={
    key_mgmt=IEEE8021X
    eap=PEAP
    identity="bob"
    password="gns3"
    phase2="auth=MSCHAPV2"
}
```

Authentication was initiated using:

```
wpa_supplicant -Dwired -ieth0 -c/usr/wpa.conf
```

Successful authentication was confirmed on the Cisco switch through the authenticated session table.



```
Switch#sh authentication sessions

Interface    MAC Address    Method  Domain  Status Fg Session ID
Gi0/0        0242.9b27.a300 dot1x    DATA   Auth   0A01016400000000C0007766D

Session count = 1
```

Figure 21: Authenticated 802.1X session on the Cisco switch

4.3.1 Wireshark Analysis of EAP-PEAP Authentication

Wireshark was used to capture and analyze the EAP-PEAP authentication exchange between the supplicant and the switch.

The captures show the complete IEEE 802.1X authentication process, including:

- EAPOL-Start
- EAP Request Identity
- EAP Response Identity
- Protected EAP (PEAP)
- TLS handshake messages
- EAP Success

1	15:18:12.336932	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAPOL	18	Start
2	15:18:12.356056	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	60	Request, Identity
3	15:18:12.356948	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAP	26	Response, Identity
4	15:18:12.404767	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	60	Request, Protected EAP (EAP-PEAP)
5	15:18:12.407037	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	218	Client Hello
6	15:18:12.442912	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	1022	Request, Protected EAP (EAP-PEAP)
7	15:18:12.443276	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAP	24	Response, Protected EAP (EAP-PEAP)
8	15:18:12.474104	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	200	Server Hello, Certificate, Server Key Exchange, Server Hello Done
9	15:18:12.477726	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	154	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
10	15:18:12.519615	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	75	Change Cipher Spec, Encrypted Handshake Message
11	15:18:12.520202	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAP	24	Response, Protected EAP (EAP-PEAP)
12	15:18:12.565761	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	60	Application Data
13	15:18:12.566196	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	57	Application Data
14	15:18:12.610079	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	92	Application Data
15	15:18:12.610534	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	111	Application Data
16	15:18:12.664969	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	100	Application Data
17	15:18:12.666643	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	55	Application Data
18	15:18:12.711178	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	64	Application Data
19	15:18:12.712360	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	64	Application Data
20	15:18:12.856284	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	60	Success

Figure 22: Wireshark capture of the EAP-PEAP authentication process

The authentication process starts when the supplicant sends an EAPOL-Start frame after connecting to the switch port.

The switch, acting as the authenticator, responds with an EAP Request Identity message. The supplicant then replies with an EAP Response Identity containing the username used for authentication.

```

> Frame 3: Packet, 26 bytes on wire (208 bits), 26 bytes captured (208 bits) on interface -, id 0
> Ethernet II, Src: 02:42:9b:27:a3:00 (02:42:9b:27:a3:00), Dst: Nearest-non-TPMR-Bridge (01:80:c2:00:00:03)
> 802.1X Authentication
  Extensible Authentication Protocol
    Code: Response (2)
    Id: 1
    Length: 8
    Type: Identity (1)
    Identity: bob

```

Figure 23: EAP Response Identity packet containing the username used during authentication

The EAP Response Identity packet contains the username provided by the supplicant during the authentication exchange.

In this setup, the supplicant identified itself using the username **bob**, which was later validated by the FreeRADIUS server through PEAP/MSCHAPv2 authentication.

The switch forwards the authentication information to the FreeRADIUS server using the RADIUS protocol.

PEAP negotiation is then initiated in order to establish a TLS tunnel between the supplicant and the authentication server. The Wireshark captures show TLS handshake messages, including Client Hello and Change Cipher Spec.

After the TLS tunnel is established, MSCHAPv2 authentication takes place inside the encrypted channel.

Finally, the authentication process completes successfully with the EAP Success message, allowing the switch to authorize the port for normal network access.

4.3.2 Security Considerations

IEEE 802.1X authentication can mitigate several network attacks studied in Lab 1.1.

By requiring authentication before granting access to switch ports, unauthorized devices cannot easily connect to the network.

This mechanism helps mitigate:

- Unauthorized network access
- Rogue devices connected to switch ports
- Basic MAC spoofing attempts
- Passive credential sniffing attacks

The use of PEAP additionally protects authentication credentials by establishing an encrypted TLS tunnel between the supplicant and the authentication server.

However, IEEE 802.1X does not directly mitigate Denial of Service attacks such as ICMP flood or TCP SYN flood attacks studied previously.

4.3.3 Comparison Between AI Explanation and Observed Traffic

An AI tool was used to explain the EAP-PEAP authentication process implemented in the lab setup.

The AI explanation correctly identified the three main entities involved in IEEE 802.1X authentication:

- Supplicant
- Authenticator
- Authentication Server

The supplicant corresponds to the Linux host running `wpa_supplicant`, which initiates the authentication process and provides user credentials.

The authenticator corresponds to the Cisco switch, which controls access to the network port and forwards EAP messages between the supplicant and the RADIUS server.

The authentication server corresponds to the FreeRADIUS server, which validates credentials and decides whether authentication succeeds or fails.

The Wireshark captures confirmed the behavior described by the AI explanation. Observed traffic included EAPOL-Start, Identity exchange messages, PEAP negotiation, TLS handshake packets, and the final EAP Success message.

One important observation is that the switch does not authenticate the user directly. Instead, the switch acts only as an intermediary between the supplicant and the FreeRADIUS server. The actual authentication decision is performed exclusively by the RADIUS server.

Overall, the AI explanation was consistent with the observed protocol behavior and packet captures.

5 Conclusion

6 Appendix